

Q3: @Transactional — Deep Internals for Architect Interviews (15-Yr Level)

Study Time: 25-30 minutes | **Frequency:** 90% in architect rounds 🔥🔥🔥 | **Difficulty:** ★★★★★

"You said @Transactional uses a proxy. Walk me through exactly what Spring creates at startup, what executes at runtime, and how the DB connection is actually bound to the thread." — Real architect round question.

BEGINNER: Start Here

New to Spring internals? Read this first. Everything is explained in plain English before the deep technical walkthrough.

Part 1 — What is CGLIB?

CGLIB = Code Generation Library. It's a Java library that creates a new class at runtime by **extending (subclassing)** your class.

Spring uses CGLIB because it needs to run extra logic (open transaction, commit, rollback) before and after your method — without you writing that code. It can't modify your class, so it creates a subclass that wraps it.

```
// You wrote this:
@Service
public class OrderService {
    @Transactional
    public void placeOrder(Order order) {
        // your business logic
    }
}

// Spring generates this at startup (you never write it, never see it):
public class OrderService$$SpringCGLIB$$0 extends OrderService {
    @Override
    public void placeOrder(Order order) {
        beginTransaction();
        try {
            super.placeOrder(order);    // YOUR actual code runs here
            commit();
        } catch (Exception e) {
            rollback();
            throw e;
        }
    }
}
```

When you `@Autowired OrderService svc`, Spring gives you the **generated subclass**, not your original. You can't tell the difference — but every method call goes through the wrapper first.

```
You call:   svc.placeOrder(order)
            |
            ▼
    OrderService$$SpringCGLIB$$0  ← the "bodyguard"
    |   beginTransaction()
    |   super.placeOrder(order)  ← YOUR actual code runs here
    |   commit() or rollback()
```

Why CGLIB and not something simpler? Java's built-in `java.lang.reflect.Proxy` can only wrap **interfaces**. CGLIB works by subclassing, so it works on any class — even ones with no interface. Spring Boot defaults to always using CGLIB.

Part 2 — Why only public methods?

CGLIB works by **overriding** your methods in the generated subclass. Java determines what a subclass can override:

Method type	Can subclass override it?	@Transactional works?
public	✓ Yes	✓ Yes

Method type	Can subclass override it?	@Transactional works?
protected	✓ Yes	✓ Yes
private	✗ No — subclass can't see it	✗ Silent skip
final	✗ No — overriding final is illegal	✗ Silent skip*

*final trap: Spring Boot 2.x silently skips. Spring Boot 3.x throws `BeanCreationException` at startup. `private` is always a silent skip in all versions — no error ever.

Part 3 — Step by Step: @Transactional in action

Say you have this code:

```
@Service
public class OrderService {

    @Autowired OrderRepository orderRepo;
    @Autowired PaymentRepository paymentRepo;

    @Transactional
    public void placeOrder(Order order) {
        orderRepo.save(order);           // SQL: INSERT INTO
orders                                     payments
        paymentRepo.save(order.getPayment()); // SQL: INSERT INTO
    }
}
```

Step 1 — Proxy intercepts the call

```
You call: orderService.placeOrder(order)
          |
          ▼
    OrderService$$CGLIB (the proxy)
    – NOT your class yet –
```

Step 2 — Proxy grabs a connection from the pool

```
Connection pool: [conn1, conn2, conn3, conn4...]
                  |
                grabs conn1
```

Step 3 — Proxy turns off autoCommit

By default in JDBC, every SQL commits immediately. Spring turns this OFF so it can control when to commit.

```
conn1.setAutoCommit(false)    ← SQL won't commit until we say so
```

Step 4 — Proxy writes conn1 on this thread's sticky note (ThreadLocal)

Where does "Thread-7" come from? You don't pick the thread — Tomcat manages a pool of threads (e.g. 200 threads named `http-nio-8080-exec-1`, `exec-2` ... `exec-200`). When your HTTP request arrives, Tomcat assigns it to whichever thread is free at that moment. It could be Thread-1 or Thread-99. "Thread-7" is just an example name — whatever thread handles your request gets its own private ThreadLocal slot.

```
HTTP request arrives
    |
    ▼
Tomcat picks a free thread (e.g. Thread-7)
    |
    ▼
Spring stores conn1 in Thread-7's ThreadLocal slot
    |
    ▼
All @Repository calls in this request → use Thread-7's slot → conn1
    |
    ▼
Request done → Thread-7 goes back to pool, ThreadLocal cleared
```

```
TransactionSynchronizationManager.bindResource(dataSource, conn1)
```

```
Thread-7's sticky note: [dataSource → conn1]
```

Step 5 — Proxy calls your actual method

```
super.placeOrder(order)    // now YOUR code runs
```

Step 6 — `orderRepo.save(order)` runs

Inside the repo, Spring asks: "give me a connection for this dataSource"

```
DataSourceUtils.getConnection(dataSource)
    → checks Thread-7's sticky note
    → finds conn1
    → uses conn1
```

```
SQL: INSERT INTO orders VALUES (...) ← runs on conn1, NOT committed yet
```

Step 7 — `paymentRepo.save(...)` runs

Same thing — also asks for a connection:

```
DataSourceUtils.getConnection(dataSource)
  → checks Thread-7's sticky note
  → finds conn1 ← SAME connection!
```

```
SQL: INSERT INTO payments VALUES (...) ← also on conn1, NOT committed yet
```

Step 8 — Your method returns normally

Control goes back to the proxy:

```
conn1.commit() ← BOTH inserts commit together atomically
Thread-7's sticky note wiped clean
conn1 returned to pool
```

If an exception is thrown instead:

```
conn1.rollback() ← BOTH inserts cancelled
Thread-7's sticky note wiped clean
conn1 returned to pool
```

Why this design? The sticky note (`ThreadLocal`) is the only way the proxy (Step 4) and the repos (Steps 6, 7) can share the same connection without passing it as a method argument. The repos don't know about `@Transactional` at all — they just ask "give me a connection" and Spring silently gives them the one already on the thread.

Part 4 — Commit / Rollback

After your method finishes **normally** → Spring commits and erases the sticky note (clean for next request).

If your method throws a `RuntimeException` → Spring rolls back and erases the sticky note.

Checked exceptions (like `IOException`) do **NOT** trigger rollback by default — only `RuntimeException` and `Error`. Use `@Transactional(rollbackFor = IOException.class)` if you need it.

Part 5 — Three Senior Traps

Trap 1 — `final` class or `final` method: CGLIB can't subclass `final`. No proxy is created → `@Transactional` does nothing. Silent skip (or startup crash in Spring Boot 3.x).

Trap 2 — Self-invocation (`this.myMethod()`):

```
public void A() {  
    this.B();    // WRONG – bypasses the proxy  
}  
  
@Transactional  
public void B() { ... }
```

`this` points to **your original class**, not the CGLIB proxy. Calling `this.B()` skips the bodyguard → no transaction opens.

Fix: inject the service into itself via `@Autowired`, or move `B()` into a separate class.

Trap 3 — `@Async`: `@Async` runs on a **new thread**. New thread = blank sticky note. No transaction from the caller's thread exists here. The two threads can never share a transaction.

Part 6 — Propagation (plain English)

REQUIRED (default):

"Is there already a sticky note on this thread? Yes → reuse it. No → create a new one."

REQUIRES_NEW:

"Ignore whatever sticky note exists. Pause it, grab a brand new connection, do my work, commit, then restore the old note."

Real use case: audit logging — you want the audit record saved **even if the main transaction rolls back**. Annotate the audit method **REQUIRES_NEW** — it commits independently.

One-Liner (memorize this)

"Spring generates a subclass proxy using CGLIB, intercepts my `@Transactional` method, stores the JDBC connection in a `ThreadLocal` so all repos in the same thread share it, then commits or rolls back at the end."

Now you're ready for the deep technical walkthrough below.

NEW LEARNER FOUNDATION — Read This First

What is a Transaction? (Plain English)

Imagine you're at an ATM transferring money:

Step 1: Subtract 500 from your account

Step 2: Add 500 to the other account

If Step 1 succeeds but Step 2 crashes (power cut, network error):

→ Your 500 is gone but never arrived. Data is corrupt.

A Transaction groups steps so they either ALL succeed or ALL undo:

COMMIT = everything worked → permanently save all changes

ROLLBACK = something failed → undo everything, as if nothing happened

@Transactional tells Spring: "wrap this method in a transaction"

What is a Proxy? (Plain English)

Spring Proxy: when you call `orderService.placeOrder()`, you don't call the real `OrderService` directly. You call a PROXY object that:

- Opens a DB transaction before your method
- Runs your real method
- Commits or rolls back after

The real `OrderService` doesn't know a proxy exists. It just runs normally.

What is ThreadLocal? (Plain English)

A `ThreadLocal` is a variable where each thread has its OWN copy.

Example: 200 users send requests at the same time = 200 threads.

Thread 1 (User A): DB connection = conn#42

Thread 2 (User B): DB connection = conn#99

Thread 3 (User C): DB connection = conn#17

Each thread has its OWN connection in the `ThreadLocal`.

Thread 1 can never accidentally use Thread 2's connection.

This is how `@Transactional` keeps 200 concurrent transactions isolated.

What is autoCommit? (Plain English)

DB default (`autoCommit=ON`): every SQL statement commits immediately.

INSERT INTO orders ... → committed instantly, can't undo

Transaction mode (`autoCommit=OFF`): statements are batched.

INSERT INTO orders ... → held, not committed yet

INSERT INTO order_items ... → held, not committed yet

```
conn.commit() → both committed together atomically
```

```
@Transactional does: conn.setAutoCommit(false) at start,  
conn.commit() at end (or rollback on exception).
```

BIG PICTURE — Where @Transactional Fits in Your App

SPRING BOOT APPLICATION

HTTP Request → Controller → Service → Repository → Database

@EnableTransactionManagement (startup – runs ONCE)

InfrastructureAdvisorAutoProxyCreator (BeanPostProc.)
For every @Service bean with @Transactional:
→ creates CGLIB proxy wrapper

Runtime – every request to a @Transactional method:

[HTTP Request]



[OrderService CGLIB Proxy] ←←← THIS FILE
TransactionInterceptor.invoke()

1. getTransaction() → doBegin()
2. conn.setAutoCommit(false)
3. ThreadLocal.bind(DataSource → conn#42)

↓
[Real OrderService.placeOrder()]

↓
[OrderRepository]

DataSourceUtils.getConnection()
→ ThreadLocal.get() → conn#42 (same TX!)

↓
[PostgreSQL / MySQL Database]

← result flows back up

↓
[Proxy resumes]

no exception → conn#42.commit() → ThreadLocal.unbind()
exception → conn#42.rollback() → ThreadLocal.unbind()

HikariCP Connection Pool (lives outside the request thread)

```
[conn#42] [conn#99] [conn#17] ...
available connections waiting
doBegin() borrows one
commit/rollback returns it
```

ThreadLocal (one per thread, isolated):

```
Thread 1: DataSource → conn#42
Thread 2: DataSource → conn#99
Thread 3: DataSource → conn#17
```

"Two main hidden areas. First, rollback: only RuntimeException triggers it by default — checked exceptions silently commit. And if you catch the exception yourself, Spring never sees it — use `setRollbackOnly()` to force a rollback. Second, proxy limitations: self-invocation via this bypasses the proxy, so `REQUIRES_NEW` silently does nothing — fix by extracting to a separate bean. Private and final methods are also silently ignored."

Tomcat thread by default. But more precisely, it's "whatever thread is currently running."

Here's how they're connected:

HTTP Request arrives | ▼ Tomcat picks Thread-7 from its thread pool | ▼ Spring `@Transactional` begins on Thread-7 | ▼ TSM stores `[dataSource → conn1]` in Thread-7's ThreadLocal slot | ▼ All code in this request runs on Thread-7 → `orderRepo`, `paymentRepo` all use Thread-7's slot → `conn1` | ▼ Request done → TSM clears Thread-7's slot → Thread-7 goes back to Tomcat pool It's not "Tomcat-specific" though. TSM uses Java's ThreadLocal — which works for ANY thread type:

Thread source TSM works? Tomcat HTTP thread (default) ✅ Yes @Scheduled thread ✅ Yes Manual new Thread(...) ✅ Yes @Async thread ✅ Yes — but a new TSM slot (empty — no transaction from caller) The key rule: one thread = one TSM slot. Tomcat just happens to be the most common source of threads in a web app.

TSM does NOT hold locks — it only holds the JDBC connection reference. The actual database lock lives in the database server (e.g. PostgreSQL row lock), not in Java/Spring. TSM just makes sure all code in the same thread uses the same connection so they participate in the same DB transaction.

Part 1: Startup — How Spring Builds the Proxy

`@EnableTransactionManagement` Registers Two Key Beans

```
// When you add this annotation (or include spring-boot-starter-data-jpa):
@EnableTransactionManagement
public class App { }

// Spring imports ProxyTransactionManagementConfiguration which registers:
```

```
// 1. InfrastructureAdvisorAutoProxyCreator
//   - a BeanPostProcessor that runs after every bean is created
//   - checks if any Advisor matches the bean
//   - if yes: wraps it in a proxy

// 2. BeanFactoryTransactionAttributeSourceAdvisor
//   - the Advisor that defines WHAT to intercept (Pointcut) and WHAT to
//   do (Advice)
//   - Pointcut: AnnotationTransactionAttributeSource
//       → scans methods for @Transactional
//   - Advice: TransactionInterceptor
//       → the actual code that starts/commits/rolls back transactions

// 3. AnnotationTransactionAttributeSource
//   - parses @Transactional attributes (propagation, isolation,
//   rollbackFor, timeout)
//   - caches the parsed TransactionAttribute per method
```

CGLIB vs JDK Proxy — Which One Gets Created?

JDK Dynamic Proxy (java.lang.reflect.Proxy):

WHEN: the bean's class implements at least one interface

HOW: creates a synthetic class implementing the same interface(s)
delegates all interface method calls to an InvocationHandler

LIMIT: can only proxy interface methods

CGLIB Proxy (Code Generation Library):

WHEN: the bean class does NOT implement an interface, OR
spring.aop.proxy-target-class=true (Spring Boot default: TRUE)

HOW: creates a SUBCLASS of your bean class at runtime
overrides public/protected methods with interception logic

LIMIT: cannot subclass final classes or override final methods
cannot intercept private methods (not visible in subclass)

Spring Boot default (spring.aop.proxy-target-class=true):

→ CGLIB for ALL beans, even if they implement interfaces

→ Consistent behavior – callers can inject the concrete type

→ CGLIB overhead: ~1-3% at startup (class generation), 0% at runtime

```
// Proof – what a CGLIB proxy looks like (conceptually):
// Spring generates something like:
public class OrderService$$SpringCGLIB$$0 extends OrderService {

    @Override
    public void placeOrder(OrderRequest req) {
        // Intercept call → find matching Advisors → build interceptor
        chain
        ReflectiveMethodInvocation chain = buildChain(advisors, this,
        method, args);
```

```

        chain.proceed();
        // chain internally calls: TransactionInterceptor → real
        OrderService.placeOrder()
    }
}
// The "real" OrderService.placeOrder() is invoked by the chain INSIDE the
// interceptor

```

Part 2: Runtime — TransactionInterceptor in Detail

The TransactionInterceptor.invoke() Flow

```

// What TransactionInterceptor actually does (simplified source
// walkthrough):

public Object invoke(MethodInvocation invocation) throws Throwable {
    // Step 1: Resolve the target class (needed for annotation lookup)
    Class<?> targetClass = invocation.getThis().getClass();
    Method method = invocation.getMethod();

    // Step 2: Get @Transactional metadata for this method
    // (propagation, isolation, timeout, rollbackFor, readOnly)
    TransactionAttribute txAttr =
txAttributeSource.getTransactionAttribute(method, targetClass);

    // Step 3: Get the PlatformTransactionManager to use
    // (could be DataSourceTransactionManager, JpaTransactionManager, etc.)
    PlatformTransactionManager tm = determineTransactionManager(txAttr);

    // Step 4: Begin/join/suspend transaction
    TransactionInfo txInfo = createTransactionIfNecessary(tm, txAttr,
joinpointIdentification);
    // This calls: tm.getTransaction(txAttr)
    // which calls: AbstractPlatformTransactionManager.getTransaction()

    Object retVal;
    try {
        // Step 5: Execute the real method
        retVal = invocation.proceed();

    } catch (Throwable ex) {
        // Step 6a: Exception path – rollback or commit based on
        rollbackFor rules
        completeTransactionAfterThrowing(txInfo, ex);
        throw ex;
    } finally {
        // Step 7: Clean up TransactionInfo from thread-local stack
        cleanupTransactionInfo(txInfo);
    }
}

```

```
// Step 6b: Success path
commitTransactionAfterReturning(txInfo);
return retVal;
}
```

Part 3: The ThreadLocal — How the DB Connection Is Bound

TransactionSynchronizationManager (The Most Important Class Nobody Mentions)

This is the CORE mechanism that makes @Transactional work.
It maintains ThreadLocal state for the current transaction:

TransactionSynchronizationManager holds (all ThreadLocal):

- resources: Map<Object, Object>
 - key: DataSource (the connection pool object)
 - value: ConnectionHolder (wraps the java.sql.Connection)
 - "The current thread has connection X for DataSource Y"
- synchronizations: Set<TransactionSynchronization>
 - registered callbacks (afterCommit, afterRollback, afterCompletion)
 - used by @TransactionalEventListener
- currentTransactionName: String
- currentTransactionReadOnly: Boolean
- actualTransactionActive: Boolean

```
// DataSourceTransactionManager.doBegin() – step by step:
protected void doBegin(Object transaction, TransactionDefinition
definition) {
    DataSourceTransactionObject txObject = (DataSourceTransactionObject)
transaction;

    // 1. Acquire connection from HikariCP pool
    Connection con = obtainDataSource().getConnection();

    // 2. Wrap in a holder
    ConnectionHolder conHolder = new ConnectionHolder(con, true);
    txObject.setConnectionHolder(conHolder, true);

    // 3. Apply transaction settings
    if (def.isReadOnly()) con.setReadOnly(true);
    if (def.getTimeout() != TIMEOUT_DEFAULT)
conHolder.setTimeoutInSeconds(def.getTimeout());

    // 4. CRITICAL: turn off auto-commit – this starts the DB transaction
    con.setAutoCommit(false);
```

```
// 5. CRITICAL: bind connection to current thread
TransactionSynchronizationManager.bindResource(observeOnDataSource(),
conHolder);
// Now: this thread = this connection = this transaction
}

// How JPA/JDBC repositories FIND the connection:
// DataSourceUtils.getConnection(dataSource)
//   → checks TransactionSynchronizationManager.getResource(dataSource)
//   → returns the same ConnectionHolder (same connection, same TX)
// This is how all your repositories in the same TX method share one
connection
```

Part 4: Rollback Rules — The Hidden Traps

Open (10 sec)

"The default is: only RuntimeException and Error trigger rollback. Checked exceptions do NOT rollback by default — this catches a lot of people."

The payment example (20 sec)

"Classic trap: imagine a payment method that throws a checked PaymentException. The repo already marked the order as 'processing' before the exception — but since it's a checked exception, Spring doesn't roll back. So the order is stuck in 'processing' with no charge ever made. Fix is @Transactional(rollbackFor = PaymentException.class) — or just make it extend RuntimeException."

The swallowed exception trap (20 sec)

"Second trap — if you catch the exception inside the method, Spring never sees it. Transaction commits even though something failed. If you still need to rollback after catching, call TransactionAspectSupport.currentTransactionStatus().setRollbackOnly() — that forces a rollback even though the exception was swallowed."

Default Rollback Behavior (Often Gets Wrong in Interviews)

```
// DEFAULT: only RuntimeException (and Error) trigger rollback
// Checked exceptions DO NOT trigger rollback unless configured

@Transactional
public void processPayment(Long orderId) throws PaymentException {
    paymentRepo.markProcessing(orderId);
    // PaymentException is a checked exception
    stripeClient.charge(orderId); // throws PaymentException
    // → NO rollback! paymentRepo.markProcessing() COMMITTED
    // → payment marked as "processing" but charge never happened
}

// FIX
@Transactional(rollbackFor = PaymentException.class)
```

```
public void processPayment(Long orderId) throws PaymentException { ... }
```

```
// Or: make PaymentException extend RuntimeException
```

```
// TRAP: Exception caught in the method prevents rollback
```

```
@Transactional
```

```
public void processOrder(Long orderId) {
```

```
    try {
```

```
        inventoryService.deduct(orderId);    // throws StockException
```

```
    } catch (StockException e) {
```

```
        log.warn("Stock issue: {}", e.getMessage());
```

```
        // Exception swallowed – @Transactional never sees it
```

```
        // TX commits even though deduction failed!
```

```
    }
```

```
    orderRepo.save(new Order(orderId));    // this saves even on stock  
failure
```

```
}
```

```
// FIX: if you catch but still want rollback:
```

```
} catch (StockException e) {
```

```
    TransactionAspectSupport.currentTransactionStatus().setRollbackOnly();
```

```
    // Marks TX as "must rollback" – commit will throw
```

```
UnexpectedRollbackException
```

```
}
```

Part 5: Proxy Limitations — The Complete List

Open (10 sec)

"The proxy has one big limitation: it only intercepts calls that come in from outside the bean. If a method calls another method on the same class using this, it bypasses the proxy entirely."

Self-invocation (25 sec)

"Example: placeOrder() calls this.notifyWarehouse(). notifyWarehouse() has REQUIRES_NEW — you'd expect a new transaction. But this points to your original class, not the CGLIB proxy. So REQUIRES_NEW is silently ignored — it runs in the same transaction. Fix: move notifyWarehouse into a separate bean, inject it, then call it through that bean — now it goes through the proxy."

Other limitations (20 sec)

"Three more: final class throws a startup exception in Spring Boot 3. final method and private method are silently ignored — @Transactional does nothing, no error. And calling a @Transactional method from @PostConstruct is risky because the proxy may not be fully wired yet during bean initialization."

Self-Invocation (Most Common)

```

@Service
public class OrderService {
    @Transactional
    public void placeOrder(OrderRequest req) {
        // Proxy intercepts this ✓

        this.notifyWarehouse(req); // ✗ THIS IS NOT THROUGH THE PROXY
        // 'this' = real OrderService (not the CGLIB proxy)
        // The proxy is the object that the ApplicationContext holds
        // Inside the method, 'this' always refers to the target, not the
proxy
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void notifyWarehouse(OrderRequest req) {
        // Runs in the SAME transaction as placeOrder, not a new one
        // REQUIRES_NEW is silently ignored
    }
}

```

```

// FIX 1: Extract to another bean (preferred)
@Service
public class WarehouseNotifier {
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void notify(OrderRequest req) { ... }
}

@Service
public class OrderService {
    @Autowired WarehouseNotifier warehouseNotifier;

    @Transactional
    public void placeOrder(OrderRequest req) {
        warehouseNotifier.notify(req); // goes through proxy ✓
    }
}

// FIX 2: Self-injection (use sparingly – creates circular dep, confusing)
@Service
public class OrderService implements ApplicationContextAware {
    private OrderService proxy; // the proxy, not 'this'

    @Override
    public void setApplicationContext(ApplicationContext ctx) {
        this.proxy = ctx.getBean(OrderService.class); // Spring gives us
the proxy
    }

    @Transactional
    public void placeOrder(OrderRequest req) {

```

```

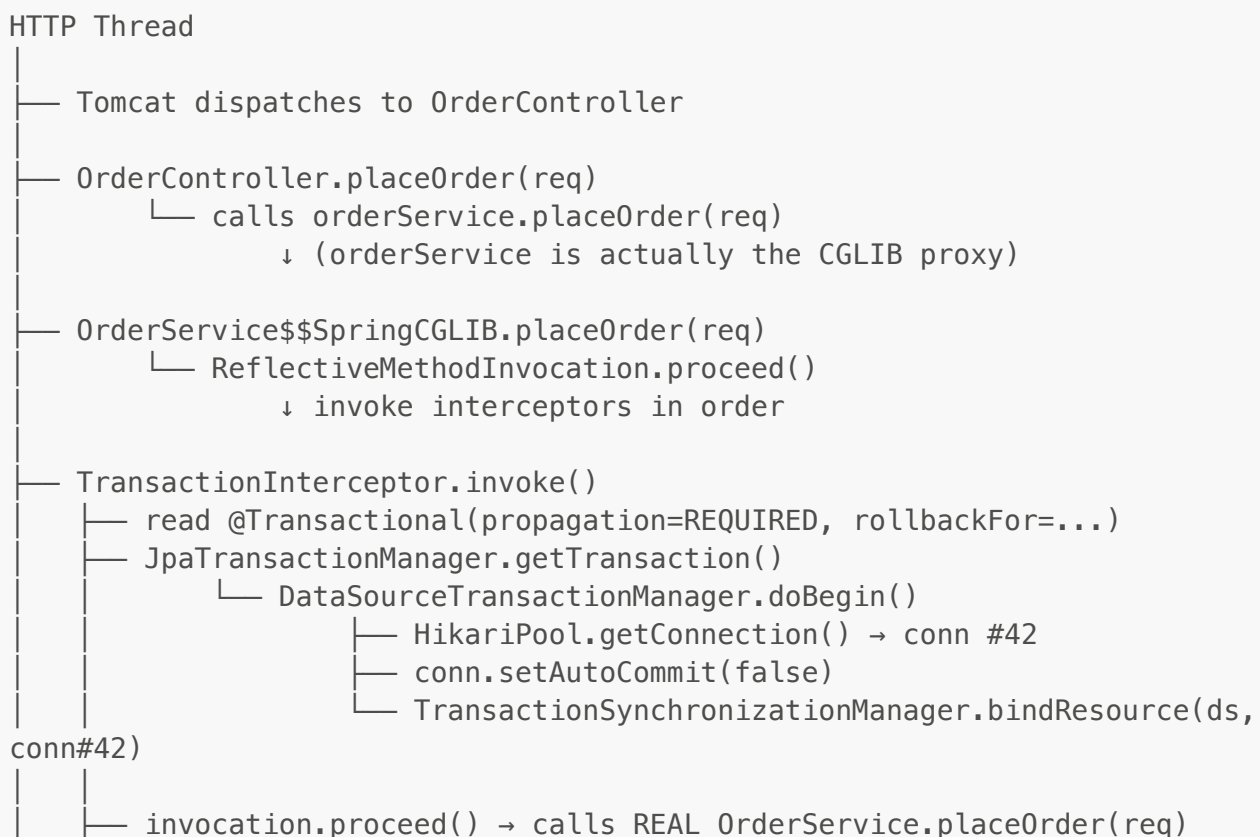
        proxy.notifyWarehouse(req); // through proxy ✅
    }
}

```

Other Proxy Limitations

- ❌ final class: CGLIB cannot subclass final classes
 - Exception at startup: "Cannot subclass final class"
- ❌ final method: CGLIB cannot override final methods
 - @Transactional on final method is SILENTLY IGNORED (no error!)
 - Worst trap: no exception, annotation just does nothing
- ❌ private method: not visible to subclass proxy
 - @Transactional silently ignored
- ❌ @Transactional on @Bean factory method (in @Configuration):
 - works differently – proxied at the @Configuration level
 - usually do NOT put @Transactional on @Bean methods
- ❌ Called from @PostConstruct:
 - bean is being initialized, proxy may not be fully wired
 - @Transactional may or may not apply depending on initialization order

Architecture Diagram: Full Runtime Call Stack



```

└─ orderRepo.save(order)
    └─ EntityManager.persist()
        └─ DataSourceUtils.getConnection(ds)
            └─ TSM.getResource(ds) → conn#42 ✓
same connection
└─ [success] JpaTransactionManager.commit()
    └─ conn#42.commit()
    └─ conn#42.setAutoCommit(true)
    └─ TSM.unbindResource(ds) ← connection returned to HikariCP
pool
└─ [failure] JpaTransactionManager.rollback()
    └─ conn#42.rollback()
    └─ TSM.unbindResource(ds)
└─ response returned to caller

```

Interview Cheat Sheet

"When @EnableTransactionManagement is present, Spring registers InfrastructureAdvisorAutoProxyCreator — a BeanPostProcessor that checks every bean against the @Transactional Advisor. For matching beans, it creates a CGLIB subclass (Spring Boot default, even if there's an interface) that overrides public methods with a ReflectiveMethodInvocation interceptor chain. At runtime, TransactionInterceptor calls AbstractPlatformTransactionManager.getTransaction(), which calls doBegin() — this acquires a connection from HikariCP, sets autoCommit=false, and binds the connection to the current thread via TransactionSynchronizationManager's ThreadLocal resource map. All repositories called on the same thread get the same connection from that ThreadLocal, making them all part of one transaction. On success: commit + unbind. On RuntimeException (or configured checked): rollback + unbind. Self-invocation bypasses the proxy because 'this' inside a method is the real object, not the CGLIB subclass held in the ApplicationContext."